

Package ‘lebesgueFDA’

October 3, 2026

Type Package

Title Functional Data Analysis under Send-on-Delta Sampling

Version 0.1.0

Description Estimation and identification for samples of curves recorded by send-on-delta (Lebesgue) sampling, under which a curve is stored only when it has moved a fixed distance from its last stored value. The package computes the records exactly, reconstructs each curve by its held, chord or midpoint path, and estimates the mean, the covariance and the functional principal components from an exact Gram matrix, without quadrature. Under a known Lipschitz bound it gives the sharp envelopes of the paths compatible with a record, the closed-form identification width, the Chebyshev center and radius of the identified set of means, and finite-sample simultaneous bands for the identified mean. Generators for the six simulation designs of the accompanying paper are included.

License GPL (≥ 3)

Encoding UTF-8

Depends R ($\geq 3.5.0$)

Imports stats

Contents

lebesgueFDA-package	2
chord_path	3
design_truth	4
held_path	5
midpoint_path	6
path_eval	7
path_inner	8
path_linear	9
predict.sod_fpca	10
sim_coefficients	11
sim_curves	11
sim_records	12
sim_sod	13
sod_band	14
sod_compatible	16
sod_cov	17

sod_eigenfunctions	18
sod_encode	19
sod_encode_curve	20
sod_encode_sample	21
sod_envelopes	22
sod_fpca	23
sod_linear_bounds	24
sod_mean	25
sod_midpoint_risk	26
sod_paths	27
sod_readings	28
sod_reconstruct	29
sod_record	30
sod_width	31
sod_width_range	32

Index	33
--------------	-----------

lebesgueFDA-package	<i>Functional Data Analysis under Send-on-Delta Sampling</i>
---------------------	--

Description

Estimation and identification for samples of curves recorded by send-on-delta (Lebesgue) sampling: a curve is stored only when it has moved a fixed distance δ from its last stored value, so the observation times are first-exit times of the curve itself.

Details

For a continuous curve x on $[0, 1]$ the event times are $\tau_0 = 0$ and

$$\tau_{k+1} = \inf [t > \tau_k : |x(t) - x(\tau_k)| \geq \delta],$$

and the record consists of $x(0)$, the event times and the directions of the moves ([sod_record](#)). A record carries information in two forms: the events fix the curve at the event times, and the silences confine it to a band of half-width δ around the last recorded value.

Smoothing the pooled recorded values of many curves, as is usual for sparse functional data, estimates the variation-weighted function $\mu_{ev}(t) = E[X(t)|X'(t)|]/E|X'(t)|$ rather than the mean. The gap equals $\text{Cov}(X(t), |X'(t)|)/E|X'(t)|$ and does not shrink as $\delta \rightarrow 0$, so the package does not pool.

Averages of the held, chord or midpoint reconstructions ([held_path](#), [chord_path](#), [midpoint_path](#)) estimate the mean and the covariance with expected squared error of order $n^{-1} + \delta^2$, which is min-imax. Principal components are computed exactly from an $n \times n$ Gram matrix ([sod_fpca](#)).

At a fixed threshold the mean is not identified. Under a known Lipschitz bound L and a bound B on $|X(0)|$, the paths compatible with a record lie between sharp envelopes ([sod_envelopes](#)) whose integrated width has a closed form ([sod_width](#)). The identified set of mean functions has an explicit Chebyshev center and radius ([sod_midpoint_risk](#)), and [sod_band](#) gives finite-sample simultaneous bands.

[sim_sod](#) and its building blocks generate the six designs of the paper; [design_truth](#) gives their population quantities.

chord_path

*Chord Path of a Record***Description**

Returns the linear interpolation of the readings of a record.

Usage

```
chord_path(record)
```

Arguments

record An object of class "sod_record".

Details

The chord path interpolates linearly between consecutive readings and keeps the last recorded value after the last event. Between two events it lies between v_k and $v_{k+1} = v_k \pm \delta$, while the curve stays within δ of v_k , so $\sup_t |q(t) - x(t)| \leq 2\delta$ for every continuous curve. Chord averages therefore attain the minimax order $n^{-1} + \delta^2$, like held-path averages. For a twice differentiable curve whose speed is at least $s > 0$ between two events, the error there is at most $\delta^2 \sup |x''|/(8s^2)$, which explains the accuracy of chord estimators for smooth curves.

Value

An object of class "sod_path": a continuous piecewise linear function.

See Also

[held_path](#), [midpoint_path](#), [sod_fpca](#)

Examples

```
## Not run:
## random lines of design D5: the chord path is exact up to the last event
set.seed(2026)
grid <- seq(0, 1, length.out = 501)
sim <- sim_sod(n = 1, design = "D5", delta = 0.05, t = grid)
rec <- sim$records[[1]]
error <- abs(path_eval(chord_path(rec), grid) - sim$curves[1, ])
max(error[grid <= max(c(0, rec$times))])
max(error)

## End(Not run)
```

design_truth

*Population Quantities of the Simulation Designs***Description**

Returns the mean, the covariance, the pooled target and the bounds of the six simulation designs of the paper.

Usage

```
design_truth(design = c("D1", "D2", "D3", "D4", "D5", "D6"))
```

Arguments

design One of "D1", "D2", "D3", "D4", "D5" and "D6".

Details

All random elements are independent.

D1 $X(t) = Z + At^{1+R}$ with $Z \sim U(-0.2, 0.2)$, $A \sim U(0.5, 1.5)$ and $R \sim Ber(1/2)$. The speed grows with the level, so the pooled target $13(t+2t^3)/[12(1+2t)]$ exceeds the mean $(t+t^2)/2$. Bounds $B = 0.2$, $L = 3$.

D2 $X(t) = Z + 0.9t$ with $Z \sim U(-0.6, 0.6)$: parallel lines with a common event schedule, for which pooling is unbiased. Bounds $B = 0.6$, $L = 1.5$.

D3 $X(t) = a_0 + a_1t + a_2 \sin(\pi t)$ with $a_0, a_2 \sim U(-1/2, 1/2)$ and $a_1 \sim U(0, 2)$: a bounded Lipschitz model with the sharp bounds $B = 1/2$ and $L = 2 + \pi/2$.

D4 $X(t) = 2t + \sin(2\pi t)/2 + \sum_{j=1}^3 \xi_j \phi_j(t)$ with $\xi_j \sim N(0, \lambda_j)$, $\lambda = (0.5, 0.3, 0.1)$ and $\phi = (1, \sqrt{2} \sin 2\pi t, \sqrt{2} \cos 2\pi t)$: a Gaussian process with a trend and a time-varying variance; the pooled target follows from Theorem 4.6.

D5 $X(t) = A + Bt$ with $A \sim N(0, 0.5^2)$ and $B \sim N(1, 0.4^2)$: Gaussian lines with pooled target $1.1578t$ (Example 4.9).

D6 The linear interpolant on $j/1000$, $j = 0, \dots, 1000$, of $\sqrt{0.4}(A_1 \cos t + B_1 \sin t) + \sqrt{0.6}(A_2 \cos 3t + B_2 \sin 3t)$ with standard normal coefficients. The mean is zero and the covariance is that of the interpolant; at $(0.10, 0.83)$ it equals -0.0502 , whereas the pooled pair smoother converges to 0.1855 (Example 4.10).

The Gaussian designs D4–D6 have no finite bounds.

Value

A list with the functions `mean(t)`, `covariance(s, t = s)` (returning a matrix) and `event_mean(t)`, the variation-weighted target μ_{ev} of event pooling (NULL for D3), the bounds B and L (Inf for D4–D6), and the name of the design.

See Also

[sim_sod](#), [sim_coefficients](#)

Examples

```
## Not run:
## the gap between the pooled target and the mean in designs D1, D4 and D5
grid <- seq(0, 1, length.out = 201)
gaps <- sapply(c("D1", "D4", "D5"), function(d) {
  truth <- design_truth(d)
  truth$event_mean(grid) - truth$mean(grid)
})
matplot(grid, gaps, type = "l", lty = 1, xlab = "t", ylab = "mu_ev - mu")
## Example 4.9: in design D5 the gap equals 0.157760 t
design_truth("D5")$event_mean(1) - 1

## End(Not run)
```

held_path	<i>Held Path of a Record</i>
-----------	------------------------------

Description

Returns the last-value reconstruction of a record.

Usage

```
held_path(record)
```

Arguments

record An object of class "sod_record".

Details

The held path is $H(t) = v_k$ for $t_k \leq t < t_{k+1}$, with $t_0 = 0$. It depends on the curve only through the record and satisfies $\sup_t |H(t) - x(t)| \leq \delta$ for every continuous curve x that produces the record (Lemma 4.1(a)). Integrals against the held path weight each recorded value by the time during which it remains current, which is why held-path averages estimate the mean, whereas pooling the readings weights them by the variation of the curve. It is the only reconstruction available in real time.

Value

An object of class "sod_path": a right-continuous step function.

See Also

[chord_path](#), [midpoint_path](#), [sod_fpca](#)

Examples

```
## Not run:
## a curve of design D1 recorded with delta = 0.05
set.seed(2026)
grid <- seq(0, 1, length.out = 501)
sim <- sim_sod(n = 1, design = "D1", delta = 0.05, t = grid)
h <- held_path(sim$records[[1]])
max(abs(path_eval(h, grid) - sim$curves[1, ])) # at most delta
plot(grid, sim$curves[1, ], type = "l", xlab = "t", ylab = "")
lines(grid, path_eval(h, grid), type = "s", col = 2)

## End(Not run)
```

midpoint_path	<i>Midpoint Path of a Record</i>
---------------	----------------------------------

Description

Returns the average of the two envelopes of a record under a Lipschitz bound.

Usage

```
midpoint_path(record, L)
```

Arguments

record	An object of class "sod_record", compatible with L.
L	Bound on the Lipschitz constant.

Details

The midpoint path $M_o = (\ell_o + u_o)/2$ produces the record itself and satisfies $\sup_t |M_o(t) - x(t)| \leq \sup_t w_o(t)/2 \leq \delta$ for every path x compatible with the record (Theorem 4.13(v)). Since the envelopes are themselves compatible, no path does better, so the midpoint is a Chebyshev center of the compatible paths in the uniform norm. Averages of midpoint paths estimate the center of the identified set of mean functions (Theorem 4.17).

Value

An object of class "sod_path".

See Also

[sod_envelopes](#), [sod_midpoint_risk](#)

Examples

```
## Not run:
## the record of Figure 1: the midpoint path reproduces the record
rec <- sim_records(cbind(a0 = 0.2, a1 = 0.5, a2 = 0.5), "D3", delta = 0.1)[[1]]
m <- midpoint_path(rec, L = 2 + pi / 2)
knots <- sort(unique(c(seq(0, 1, length.out = 1001), m$breaks)))
again <- sod_encode(knots, path_eval(m, knots), delta = 0.1)
all.equal(again$times, rec$times)

## End(Not run)
```

path_eval

*Evaluate a Path***Description**

Evaluates a reconstruction, an estimated mean, an estimated eigenfunction or any other object of class "sod_path".

Usage

```
path_eval(path, t)
```

Arguments

path	An object of class "sod_path".
t	Evaluation times in $[0, 1]$.

Details

Pieces are closed on the left, so held paths are right-continuous; the value at $t = 1$ is stored separately.

Value

A numeric vector of values at t .

See Also

[path_linear](#), [held_path](#)

Examples

```
## Not run:
rec <- sod_record(0, times = c(0.2, 0.5), polarity = c(1, -1), delta = 0.1)
path_eval(held_path(rec), seq(0, 1, by = 0.1))
path_eval(chord_path(rec), seq(0, 1, by = 0.1))

## End(Not run)
```

path_inner

Exact Inner Product of Two Paths

Description

Computes the integral of the product of two paths over an interval exactly.

Usage

```
path_inner(f, g, lower = 0, upper = 1)
```

Arguments

`f, g` Objects of class "sod_path".
`lower, upper` Limits of integration, with $0 \leq \text{lower} < \text{upper} \leq 1$.

Details

On a piece of length d on which both paths are affine, with midpoint m ,

$$\int fg = d f(m)g(m) + d^3 f' g' / 12.$$

Summing over the merged breakpoints gives the integral without quadrature error (Proposition 3.1(iii)).

Value

A number.

See Also

[path_linear](#), [sod_fpca](#)

Examples

```
## Not run:
p <- path_linear(c(0, 1), c(0, 1))
path_inner(p, p)
set.seed(2026)
sim <- sim_sod(n = 2, design = "D1", delta = 0.05)
path_inner(held_path(sim$records[[1]]), held_path(sim$records[[2]]), 0.1, 0.9)

## End(Not run)
```

path_linear

Continuous Piecewise Linear Path

Description

Creates the piecewise linear path through given points, in the representation used for reconstructions and estimates.

Usage

```
path_linear(knots, values)
```

Arguments

knots	Strictly increasing times, starting at 0 and ending at 1.
values	Values at the knots.

Details

A path of class "sod_path" is a function on $[0, 1]$ stored through its breakpoints, the value at the left end and the slope of each piece, and its value at 1. Reconstructions, estimated means and estimated eigenfunctions are paths of this kind, and inner products between paths are computed exactly by [path_inner](#).

Value

An object of class "sod_path".

See Also

[path_eval](#), [path_inner](#)

Examples

```
## Not run:
## a curve of design D6 is piecewise linear on 1001 points, so its integrated
## squared distance from the held path can be computed exactly
set.seed(2026)
grid <- seq(0, 1, length.out = 1001)
sim <- sim_sod(n = 1, design = "D6", delta = 0.05, t = grid)
curve <- path_linear(grid, sim$curves[1, ])
h <- held_path(sim$records[[1]])
path_inner(curve, curve) - 2 * path_inner(curve, h) + path_inner(h, h)

## End(Not run)
```

predict.sod_fpca	<i>Scores and Reconstructions of New Records</i>
------------------	--

Description

Projects new records on the mean and the eigenfunctions estimated by [sod_fpca](#).

Usage

```
## S3 method for class sod_fpca
predict(object, newdata, npc = object$npc, t = NULL, ...)
```

Arguments

object	An object of class "sod_fpca".
newdata	A list of "sod_record" objects recorded with the training threshold.
npc	Number of components used, at most object\$npc.
t	Optional evaluation times in $[0, 1]$ for the fitted curves.
...	Not used.

Details

Each new record is reconstructed with the training method, giving a path Y . Its scores are $\hat{\xi}_j = \langle Y - \hat{\mu}, \hat{\phi}_j \rangle$, computed exactly, and the fitted curve is $\hat{\mu} + \sum_{j \leq K} \hat{\xi}_j \hat{\phi}_j$.

Value

A list with the matrix scores, the fitted curves as a list of paths `paths` and, when `t` is given, their values `values`, with one row per record.

See Also

[sod_fpca](#)

Examples

```
## Not run:
## 150 test curves, as in experiment E1 of the paper
set.seed(2026)
grid <- seq(0.1, 0.9, length.out = 201)
train <- sim_sod(n = 300, design = "D4", delta = 0.05)
test <- sim_sod(n = 150, design = "D4", delta = 0.05, t = grid)
fit <- sod_fpca(train$records, method = "chord")
pred <- predict(fit, test$records, t = grid)
head(pred$scores)
## mean integrated squared reconstruction error on [0.1, 0.9]
mean(0.8 * rowMeans((pred$values - test$curves)^2))

## End(Not run)
```

sim_coefficients	<i>Random Coefficients of Simulated Curves</i>
------------------	--

Description

Draws the random coefficients that define the latent curves of a simulation design.

Usage

```
sim_coefficients(n, design = c("D1", "D2", "D3", "D4", "D5", "D6"))
```

Arguments

n	Number of curves.
design	One of "D1", ..., "D6"; see design_truth .

Details

The columns are (Z, A, R) for D1, Z for D2, (a_0, a_1, a_2) for D3, (ξ_1, ξ_2, ξ_3) for D4, (A, B) for D5 and (A_1, B_1, A_2, B_2) for D6. Separating the draw of the coefficients from the recording lets several thresholds be compared on the same curves, as in the paper.

Value

A matrix with one row per curve and named columns.

See Also

[sim_records](#), [sim_curves](#), [sim_sod](#)

Examples

```
## Not run:
set.seed(2026)
cf <- sim_coefficients(n = 300, design = "D4")
head(cf)
apply(cf, 2, var)

## End(Not run)
```

sim_curves	<i>Latent Curves Evaluated on a Grid</i>
------------	--

Description

Evaluates the latent curves defined by simulated coefficients.

Usage

```
sim_curves(coefficients, design = c("D1", "D2", "D3", "D4", "D5", "D6"), t)
```

Arguments

coefficients	Matrix from sim_coefficients .
design	The design of the coefficients.
t	Evaluation times in $[0, 1]$.

Details

The curves are those of [design_truth](#); for D6 they are the linear interpolants on the grid of 1001 points. They serve as the fully observed benchmark of the paper.

Value

A matrix with one row per curve and one column per time.

See Also

[sim_coefficients](#), [sim_sod](#)

Examples

```
## Not run:
## twenty curves of each design with the population mean
set.seed(2026)
grid <- seq(0, 1, length.out = 201)
op <- par(mfrow = c(2, 3))
for (d in c("D1", "D2", "D3", "D4", "D5", "D6")) {
  x <- sim_curves(sim_coefficients(20, d), d, grid)
  matplot(grid, t(x), type = "l", lty = 1, col = "grey50", main = d,
           xlab = "t", ylab = "")
  lines(grid, design_truth(d)$mean(grid), lwd = 2)
}
par(op)

## End(Not run)
```

sim_records

Send-on-Delta Records of Simulated Curves

Description

Records the latent curves defined by simulated coefficients with a given threshold.

Usage

```
sim_records(coefficients, design = c("D1", "D2", "D3", "D4", "D5", "D6"), delta)
```

Arguments

coefficients	Matrix from sim_coefficients .
design	The design of the coefficients.
delta	Threshold $\delta > 0$.

Details

The records are computed as in the paper: in closed form for the monotone curves of D1, D2 and D5, by bisection on monotone pieces with analytic turning points for D3 and D4 ([sod_encode_curve](#)), and exactly for the piecewise linear curves of D6 ([sod_encode](#)). The same coefficients can be recorded with several thresholds.

Value

A list of "sod_record" objects, one per curve.

See Also

[sim_coefficients](#), [sim_sod](#)

Examples

```
## Not run:
## the same 300 curves of design D4 recorded at the three thresholds of the
## paper, as in experiment E1
set.seed(2026)
cf <- sim_coefficients(n = 300, design = "D4")
truth <- design_truth("D4")
grid <- seq(0.1, 0.9, length.out = 201)
for (delta in c(0.10, 0.05, 0.025)) {
  fit <- sod_fpca(sim_records(cf, "D4", delta), method = "held", npc = 3)
  ise <- 0.8 * mean((sod_mean(fit, grid) - truth$mean(grid))^2)
  cat("delta =", delta, " integrated squared error:", signif(ise, 3), "\n")
}

## End(Not run)
```

sim_sod

Simulate Send-on-Delta Records from the Designs of the Paper

Description

Draws latent curves from one of the six simulation designs of the paper and records them with a given threshold.

Usage

```
sim_sod(n, design = c("D1", "D2", "D3", "D4", "D5", "D6"), delta = 0.05, t = NULL)
```

Arguments

n	Number of curves.
design	One of "D1", ..., "D6"; see design_truth .
delta	Threshold $\delta > 0$.
t	Optional evaluation times; when given, the latent curves on t are returned as well.

Details

The simulations of the paper use $n = 100, 300$ and 900 curves and the thresholds $\delta = 0.10, 0.05$ and 0.025 ; losses are computed on $[0.1, 0.9]$. The function combines [sim_coefficients](#), [sim_records](#) and [sim_curves](#).

Value

A list with the records, the coefficients, the population quantities truth (see [design_truth](#)), the design, the threshold delta and, when t is given, the matrix curves of latent curves.

See Also

[design_truth](#), [sod_fpca](#), [sod_band](#)

Examples

```
## Not run:
## one replication of experiment E1 for design D1, n = 300, delta = 0.05
set.seed(2026)
grid <- seq(0.1, 0.9, length.out = 201)
sim <- sim_sod(n = 300, design = "D1", delta = 0.05, t = grid)
fits <- list(held = sod_fpca(sim$records, method = "held"),
            chord = sod_fpca(sim$records, method = "chord"),
            midpoint = sod_fpca(sim$records, method = "midpoint", L = sim$truth$L))
ise <- function(m) 0.8 * mean((m - sim$truth$mean(grid))^2)
c(sapply(fits, function(f) ise(sod_mean(f, grid))), oracle = ise(colMeans(sim$curves)))

## End(Not run)
```

sod_band

Simultaneous Band for the Identified Mean Function

Description

Computes a finite-sample simultaneous band for the identified mean function under known bounds on the initial value and on the Lipschitz constant, with the estimated identification width and Chebyshev radius.

Usage

```
sod_band(records, B, L, alpha = 0.05, grid = seq(0, 1, length.out = 101))

## S3 method for class sod_band
print(x, ...)
```

Arguments

records	A list of "sod_record" objects with a common threshold.
B	Bound on $ X(0) $.
L	Bound on the Lipschitz constant.
alpha	Level; the coverage is at least $1 - \alpha$.

grid	Increasing grid from 0 to 1 used for the union bound.
x	An object of class "sod_band".
...	Not used.

Details

Let $\bar{\ell}_n$ and $\bar{u}_n$ be the averages of the envelopes of the n records ([sod_envelopes](#)), N the number of grid points and h_{max} the largest grid spacing. If the curves are independent with $|X(0)| \leq B$ and Lipschitz constant at most L , then with probability at least $1 - \alpha$,

$$\bar{\ell}_n(t) - r_n - Lh_{max} \leq E\ell_O(t) \leq Eu_O(t) \leq \bar{u}_n(t) + r_n + Lh_{max}$$

for all t simultaneously, with $r_n = (B + L)\sqrt{2\log(2N/\alpha)/n}$ (Theorem 4.21). At the grid points the term Lh_{max} can be dropped. The interval $[E\ell_O(t), Eu_O(t)]$ is the identified interval of the mean: it contains the mean of every law that produces the same record law. Its width is set by the threshold, the Lipschitz bound and the timing of the events and is at most 2δ , but it does not shrink as n grows, so the band is conservative. The bounds B and L must come from outside the data: records can contradict bounds that are too small but cannot confirm them.

Value

An object of class "sod_band": a list with the band on the grid (lower, upper), the band as paths valid for all times (lower_path, upper_path), the narrower band valid at the grid points (lower_grid, upper_grid), the mean envelopes mean_lower and mean_upper, their midpoint center, the estimated Chebyshev radius of the identified set, the mean integrated width with a $1 - \alpha$ interval width_interval (Theorem 4.15(v)), the widths of the records record_widths, the margins sampling_margin and mesh_margin, and the settings.

See Also

[sod_envelopes](#), [sod_midpoint_risk](#), [sod_linear_bounds](#)

Examples

```
## Not run:
## design D3 with n = 300, delta = 0.025 and a grid of 201 points (experiment E3)
set.seed(2026)
sim <- sim_sod(n = 300, design = "D3", delta = 0.025)
grid <- seq(0, 1, length.out = 201)
band <- sod_band(sim$records, B = sim$truth$B, L = sim$truth$L, grid = grid)
band
all(band$lower <= sim$truth$mean(grid) & sim$truth$mean(grid) <= band$upper)
plot(grid, band$upper, type = "l", ylim = range(band$lower, band$upper),
      xlab = "t", ylab = "")
lines(grid, band$lower)
lines(grid, sim$truth$mean(grid), lty = 2)
## identification widths for assumed bounds of 1, 1.5 and 2 times the valid bound
sapply(c(1, 1.5, 2), function(k)
  sod_band(sim$records, B = sim$truth$B, L = k * sim$truth$L, grid = grid)$width)

## End(Not run)
```

sod_compatible

*Compatibility of a Record with Lipschitz and Initial-Value Bounds***Description**

Checks whether a record could have been produced by a curve whose Lipschitz constant is at most L and whose initial value is at most B in absolute value.

Usage

```
sod_compatible(record, L, B = NULL)
```

Arguments

record	An object of class "sod_record".
L	Bound on the Lipschitz constant.
B	Bound on $ x(0) $, or NULL to check L only.

Details

A curve with Lipschitz constant at most L needs a time of at least δ/L to move by δ , so its record satisfies $\delta \leq L(t_k - t_{k-1})$ for every event (Theorem 4.13(i)). A record that violates this inequality, or has $|x(0)| > B$, shows that the bounds are too small; a compatible record cannot confirm that they hold.

Value

TRUE or FALSE.

See Also

[sod_envelopes](#), [sod_band](#)

Examples

```
## Not run:
## design D3 has the valid bounds B = 0.5 and L = 2 + pi/2
set.seed(2026)
sim <- sim_sod(n = 300, design = "D3", delta = 0.05)
all(sapply(sim$records, sod_compatible, L = sim$truth$L, B = sim$truth$B))
## half the bound is contradicted by some records
mean(sapply(sim$records, sod_compatible, L = sim$truth$L / 2))

## End(Not run)
```

sod_cov

*Estimated Covariance Function***Description**

Evaluates the covariance function estimated by [sod_fpca](#) at all pairs of times.

Usage

```
sod_cov(fit, s, t = s)
```

Arguments

fit	An object of class "sod_fpca".
s, t	Evaluation times in $[0, 1]$.

Details

The estimate is the sample covariance of the reconstructed paths, with divisor n ; it is the full estimate, not its expansion in the retained components. Its expected squared Hilbert–Schmidt error is of order $n^{-1} + \delta^2$ (Theorem 4.18).

Value

A matrix with `length(s)` rows and `length(t)` columns.

See Also

[sod_fpca](#), [sod_eigenfunctions](#)

Examples

```
## Not run:
## design D6: at the pair (0.10, 0.83) the covariance is -0.0502, while the
## pooled pair smoother would converge to +0.1855 (Example 4.10); estimates from
## 20 replications with n = 300 and delta = 0.05 scatter around the negative value
set.seed(2026)
est <- replicate(20, {
  sim <- sim_sod(n = 300, design = "D6", delta = 0.05)
  sod_cov(sod_fpca(sim$records, method = "held", npc = 1), 0.10, 0.83)
})
mean(est)
design_truth("D6")$covariance(0.10, 0.83)
## the estimated covariance surface of one replication
sim <- sim_sod(n = 300, design = "D6", delta = 0.05)
fit <- sod_fpca(sim$records, method = "held")
grid <- seq(0, 1, length.out = 51)
persp(grid, grid, sod_cov(fit, grid), theta = 30, phi = 30, xlab = "s",
      ylab = "t", zlab = "")

## End(Not run)
```

sod_eigenfunctions *Estimated Eigenfunctions*

Description

Evaluates the eigenfunctions estimated by [sod_fpca](#).

Usage

```
sod_eigenfunctions(fit, t)
```

Arguments

fit	An object of class "sod_fpca".
t	Evaluation times in $[0, 1]$.

Details

The eigenfunctions are exact linear combinations of the centered reconstructed paths. The sign of each is chosen so that its largest value in absolute terms is positive. For a positive and simple eigenvalue λ_j with spectral gap g_j , $\|\hat{\phi}_j - \phi_j\| \leq 2^{3/2} \|\hat{C} - C\|_{HS} / g_j$, after aligning signs (Corollary 4.19).

Value

A matrix with one row per time and one column per retained component.

See Also

[sod_fpca](#)

Examples

```
## Not run:
## design D4, whose eigenfunctions are 1, sqrt(2) sin(2 pi t) and
## sqrt(2) cos(2 pi t), with eigenvalues 0.5, 0.3 and 0.1
set.seed(2026)
sim <- sim_sod(n = 300, design = "D4", delta = 0.05)
fit <- sod_fpca(sim$records, method = "chord", npc = 3)
fit$eigenvalues
grid <- seq(0, 1, length.out = 201)
phi <- sod_eigenfunctions(fit, grid)
truth <- cbind(1, sqrt(2) * sin(2 * pi * grid), sqrt(2) * cos(2 * pi * grid))
matplot(grid, phi, type = "l", lty = 1, xlab = "t", ylab = "eigenfunctions")
matlines(grid, truth, lty = 2)

## End(Not run)
```

sod_encode

*Exact Send-on-Delta Record of a Piecewise Linear Curve***Description**

Computes the send-on-delta record of the linear interpolant of a curve given on a grid, without approximation (Algorithm S1 of the paper).

Usage

```
sod_encode(t, x, delta)
```

Arguments

t	Strictly increasing grid of times, starting at 0 and ending at 1.
x	Values of the curve at t.
delta	Threshold $\delta > 0$.

Details

On each grid segment the linear interpolant is monotone, so its events are the successive times at which it reaches the stored value plus or minus δ , and each time solves a linear equation. The record is computed in $O(G + m)$ operations for G segments and m events. It is the record of the interpolant: for an unknown signal, an excursion of height δ completed between two grid points is not seen. The function records design D6, whose curves are piecewise linear by construction.

Value

An object of class "sod_record".

See Also

[sod_encode_sample](#), [sod_encode_curve](#), [sod_record](#)

Examples

```
## Not run:
## a curve of design D6: the linear interpolant on 1001 points of a smooth
## stationary Gaussian template
set.seed(2026)
knots <- seq(0, 1, length.out = 1001)
a <- rnorm(4)
x <- sqrt(0.4) * (a[1] * cos(knots) + a[2] * sin(knots)) +
  sqrt(0.6) * (a[3] * cos(3 * knots) + a[4] * sin(3 * knots))
rec <- sod_encode(knots, x, delta = 0.05)
rec
## the held path stays within delta of the curve
max(abs(path_eval(held_path(rec), knots) - x))
plot(knots, x, type = "l", xlab = "t", ylab = "")
lines(knots, path_eval(held_path(rec), knots), type = "s", col = 2)

## End(Not run)
```

sod_encode_curve

*Send-on-Delta Record of a Curve with Known Turning Points***Description**

Computes the record of a curve that is monotone between known turning points; the event times are located by bisection.

Usage

```
sod_encode_curve(curve, turning, delta, tol = 1e-13)
```

Arguments

curve	A vectorized function on $[0, 1]$.
turning	Increasing vector of turning points, starting at 0 and ending at 1. It must contain every point at which the curve changes direction.
delta	Threshold $\delta > 0$.
tol	Tolerance of the bisection, in time.

Details

On a monotone piece the curve crosses the levels $v + \delta, v + 2\delta, \dots$ (or the corresponding decreasing levels) in order, where v is the value held at the start of the piece. All crossings of a piece are found together by bisection to the tolerance `tol`, and a level reached at the end of a piece is recorded there. Turning points are not inferred from the curve. Designs D3 and D4 of the paper are recorded in this way, with turning points computed analytically.

Value

An object of class "sod_record".

See Also

[sod_encode](#), [sim_records](#)

Examples

```
## Not run:
## the curve of Figure 1:  $X(t) = 0.2 + 0.5 t + 0.5 \sin(\pi t)$ , from design D3,
## which turns at  $t = \arccos(-1/\pi)/\pi$ 
f <- function(t) 0.2 + 0.5 * t + 0.5 * sin(pi * t)
rec <- sod_encode_curve(f, turning = c(0, acos(-1 / pi) / pi, 1), delta = 0.1)
rec
sod_readings(rec)

## End(Not run)
```

sod_encode_sample	<i>Send-on-Delta Records of Curves Observed on a Common Grid</i>
-------------------	--

Description

Applies [sod_encode](#) to each row of a matrix of curve values.

Usage

```
sod_encode_sample(t, X, delta)
```

Arguments

t	Strictly increasing grid of times, starting at 0 and ending at 1.
X	Matrix of curve values, one row per curve and one column per time.
delta	Threshold $\delta > 0$.

Details

Each record is the exact record of the linear interpolant of a row of X; see [sod_encode](#).

Value

A list of "sod_record" objects, one per row of X.

See Also

[sod_encode](#), [sim_records](#)

Examples

```
## Not run:
## curves of design D6 are piecewise linear on 1001 points, so encoding them
## from the grid gives exactly the records produced by sim_records()
set.seed(2026)
knots <- seq(0, 1, length.out = 1001)
cf <- sim_coefficients(n = 100, design = "D6")
recs <- sod_encode_sample(knots, sim_curves(cf, "D6", knots), delta = 0.05)
same <- sim_records(cf, "D6", delta = 0.05)
all.equal(lapply(recs, function(r) r$times), lapply(same, function(r) r$times))

## End(Not run)
```

sod_envelopes

*Envelopes of the Paths Compatible with a Record***Description**

Under a known Lipschitz bound, computes the pointwise smallest and largest values of the paths that produce a record, their midpoint and their distance.

Usage

```
sod_envelopes(record, L)
```

Arguments

record	An object of class "sod_record", compatible with L (see sod_compatible).
L	Bound on the Lipschitz constant.

Details

For t between two events t_k and t_{k+1} ,

$$\ell_o(t) = \max[v_k - \delta, v_k - L(t - t_k), v_{k+1} - L(t_{k+1} - t)],$$

$$u_o(t) = \min[v_k + \delta, v_k + L(t - t_k), v_{k+1} + L(t_{k+1} - t)],$$

and after the last event the third terms are dropped. These are the extremal L -Lipschitz functions through the recorded values (McShane extensions), truncated by the silence bands. By Theorem 4.13, $\ell_o(t)$ and $u_o(t)$ are the infimum and the supremum of $x(t)$ over the L -Lipschitz paths x that produce the record. The width $w_o = u_o - \ell_o$ is at most 2δ , and its integral is given in closed form by [sod_width](#). All intersections of the affine pieces are breakpoints, so the envelopes are exact.

Value

A list with the paths lower, upper, midpoint and width (objects of class "sod_path") and the number integrated_width.

See Also

[midpoint_path](#), [sod_width](#), [sod_band](#)

Examples

```
## Not run:
## Figure 1(b) of the paper: design D3 with a0 = 0.2, a1 = 0.5 and a2 = 0.5,
## threshold 0.1 and Lipschitz bound 2 + pi/2
rec <- sim_records(cbind(a0 = 0.2, a1 = 0.5, a2 = 0.5), "D3", delta = 0.1)[[1]]
env <- sod_envelopes(rec, L = 2 + pi / 2)
env$integrated_width                                # 0.1290
grid <- seq(0, 1, length.out = 1001)
low <- path_eval(env$lower, grid)
up <- path_eval(env$upper, grid)
plot(grid, up, type = "l", ylim = range(low, up), xlab = "t", ylab = "")
lines(grid, low)
```

```

lines(grid, path_eval(env$midpoint, grid), lty = 2, col = 2)
lines(grid, 0.2 + 0.5 * grid + 0.5 * sin(pi * grid), col = 4)

## End(Not run)

```

sod_fpca	<i>Mean, Covariance and Principal Components from Send-on-Delta Records</i>
----------	---

Description

Estimates the mean function, the covariance function and the functional principal components from send-on-delta records, through the held, chord or midpoint reconstructions and an exact Gram matrix.

Usage

```

sod_fpca(records, method = c("held", "chord", "midpoint"), L = NULL,
          npc = NULL, fve = 0.99, tol = 1e-10)

## S3 method for class sod_fpca
print(x, ...)

```

Arguments

records	A list of "sod_record" objects with a common threshold.
method	Reconstruction: "held", "chord" or "midpoint".
L	Bound on the Lipschitz constant; needed for "midpoint".
npc	Number of components; if NULL, chosen by fve.
fve	Fraction of variance explained, used when npc is NULL.
tol	Eigenvalues below tol times the largest are treated as zero.
x	An object of class "sod_fpca".
...	Not used.

Details

Each record O_i is reconstructed by a path Y_i , and

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n Y_i, \quad \hat{C} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{\mu}) \otimes (Y_i - \hat{\mu}).$$

The positive eigenvalues of $\hat{C}$ coincide with those of the Gram matrix $G = (n^{-1} \langle Z_i, Z_j \rangle)$ of the centered paths $Z_i = Y_i - \hat{\mu}$. An eigenvector v of G with eigenvalue λ gives the eigenfunction $(n\lambda)^{-1/2} \sum_i v_i Z_i$ and the scores $(n\lambda)^{1/2} v_i$ (Proposition 3.1). The paths are piecewise affine, so G is computed exactly and no quadrature error enters.

The held path is within δ of the curve and the chord path within 2δ , for any continuous curve. If $E\|X\|^2 \leq M$, the held mean satisfies $E\|\hat{\mu} - \mu\|^2 \leq 2\delta^2 + 2M/n$, and the covariance has expected squared Hilbert–Schmidt error of order $n^{-1} + \delta^2$; no estimator does better uniformly (Theorem 4.18). If $\delta = o(n^{-1/2})$, the estimators are asymptotically equivalent to those computed

from fully observed curves (Theorem 4.20). Chord estimators satisfy the same bounds with 2δ in place of δ and are more accurate for smooth curves. The midpoint estimator of the mean has worst-case risk $\rho^2 + \sigma_M^2/n$ over the laws that produce the same record law (Theorem 4.17; see [sod_midpoint_risk](#)).

Smoothing the pooled recorded values instead would estimate $\mu_{ev}(t) = E[X(t)|X'(t)]/E|X'(t)|$ (Theorem 4.3), which differs from the mean whenever level and speed are correlated.

Value

An object of class "sod_fpca": a list with the mean path mean, the retained eigenvalues, all positive eigenvalues values, the eigenfunctions (a list of paths), the matrix of scores, the number of components npc, the fraction of variance explained fve, the Gram matrix gram, the reconstructed paths, the number of events of each record events, and the settings method, delta and L. The covariance returned by [sod_cov](#) is the full sample covariance, not its truncated expansion.

See Also

[sod_mean](#), [sod_cov](#), [sod_eigenfunctions](#), [predict.sod_fpca](#)

Examples

```
## Not run:
## design D4 with n = 300 and delta = 0.05 (experiment E1)
set.seed(2026)
sim <- sim_sod(n = 300, design = "D4", delta = 0.05)
fit <- sod_fpca(sim$records, method = "held", npc = 3)
fit
## integrated squared errors on [0.1, 0.9], as in the paper
grid <- seq(0.1, 0.9, length.out = 201)
0.8 * mean((sod_mean(fit, grid) - sim$truth$mean(grid))^2)
0.64 * mean((sod_cov(fit, grid) - sim$truth$covariance(grid))^2)

## midpoint estimator for the bounded design D3 with its valid bound
sim3 <- sim_sod(n = 300, design = "D3", delta = 0.05)
fit3 <- sod_fpca(sim3$records, method = "midpoint", L = sim3$truth$L)
fit3$eigenvalues

## End(Not run)
```

sod_linear_bounds

Bounds for a Nonnegative Linear Functional of the Mean

Description

Estimates the endpoints of the identified interval of $\int_0^1 \psi(t)\mu(t) dt$ for a nonnegative weight function ψ .

Usage

```
sod_linear_bounds(band, weight)
```

Arguments

band	An object of class "sod_band".
weight	A nonnegative object of class "sod_path", for instance from path_linear .

Details

For nonnegative ψ , the identified set of $\int \psi \mu$ contains the open interval between $E \int \psi \ell_O$ and $E \int \psi u_O$ and is contained in its closure (Theorem 4.13(iii)). The function replaces the expectations by averages over the records, using the mean envelopes of the band. The results are estimated identification bounds, not confidence limits.

Value

A named vector with the lower and the upper bound.

See Also

[sod_band](#)

Examples

```
## Not run:
## the integral of the mean in design D3, whose true value is 1/2
set.seed(2026)
sim <- sim_sod(n = 300, design = "D3", delta = 0.05)
band <- sod_band(sim$records, B = sim$truth$B, L = sim$truth$L)
sod_linear_bounds(band, path_linear(c(0, 1), c(1, 1)))
## a weight that increases linearly over [0, 1]
sod_linear_bounds(band, path_linear(c(0, 1), c(0, 2)))

## End(Not run)
```

sod_mean	<i>Estimated Mean Function</i>
----------	--------------------------------

Description

Evaluates the mean function estimated by [sod_fpca](#).

Usage

```
sod_mean(fit, t)
```

Arguments

fit	An object of class "sod_fpca".
t	Evaluation times in $[0, 1]$.

Details

The estimate is the average of the reconstructed paths; see [sod_fpca](#) for its risk.

Value

A numeric vector of the estimated mean at t .

See Also

[sod_fpca](#), [sod_cov](#)

Examples

```
## Not run:
## design D1, where level and speed are associated: reconstruction-based
## means follow mu, whereas pooling would follow mu_ev (dashed)
set.seed(2026)
sim <- sim_sod(n = 900, design = "D1", delta = 0.025)
held <- sod_fpca(sim$records, method = "held")
chord <- sod_fpca(sim$records, method = "chord")
grid <- seq(0.1, 0.9, length.out = 201)
plot(grid, sim$truth$event_mean(grid), type = "l", lty = 2, xlab = "t",
      ylab = "mean")
lines(grid, sim$truth$mean(grid), lwd = 2)
lines(grid, sod_mean(held, grid), col = 2)
lines(grid, sod_mean(chord, grid), col = 3)

## End(Not run)
```

sod_midpoint_risk

Chebyshev Radius and Worst-Case Risk of the Midpoint Estimator

Description

Estimates the Chebyshev radius of the identified set of mean functions and the worst-case risk of the averaged midpoint estimator.

Usage

```
sod_midpoint_risk(records, L)
```

Arguments

records	A list of at least two "sod_record" objects with a common threshold, compatible with L.
L	Bound on the Lipschitz constant.

Details

With $a = E\ell_O$, $b = Eu_O$ and $c = (a + b)/2$, the closure of the identified set of mean functions has Chebyshev center c and radius $\rho = \|b - a\|/2$ in L^2 , and the average $\hat{\mu}_M$ of the midpoint paths has worst-case risk

$$\sup E\|\hat{\mu}_M - \mu\|^2 = \rho^2 + \sigma_M^2/n, \quad \sigma_M^2 = E\|M_O - c\|^2,$$

the supremum being over the laws that produce the same record law. No estimator improves on ρ^2 uniformly, and $\rho^2 \leq \delta EW(O)/2 \leq \delta^2$ (Theorem 4.17). The function replaces a , b and σ_M^2 by their sample versions; all integrals are exact.

Value

A list with the estimated radius, the sample variance of the midpoint paths, the plug-in worst-case risk risk, and risk_unbiased, which corrects the plug-in estimate of ρ^2 for its bias.

See Also

[midpoint_path](#), [sod_band](#)

Examples

```
## Not run:
## design D1 with n = 300 at the three thresholds of the paper (experiment E3)
set.seed(2026)
cf <- sim_coefficients(300, "D1")
L <- design_truth("D1")$L
sapply(c(0.10, 0.05, 0.025), function(delta)
  unlist(sod_midpoint_risk(sim_records(cf, "D1", delta), L = L)))

## End(Not run)
```

sod_paths

Reconstructed Paths of a Sample of Records

Description

Reconstructs every record of a sample by its held, chord or midpoint path.

Usage

```
sod_paths(records, method = c("held", "chord", "midpoint"), L = NULL)
```

Arguments

records	A list of "sod_record" objects with a common threshold.
method	Reconstruction: "held", "chord" or "midpoint".
L	Bound on the Lipschitz constant; needed for "midpoint".

Details

See [held_path](#), [chord_path](#) and [midpoint_path](#) for the three reconstructions and their guarantees.

Value

A list of "sod_path" objects.

See Also

[sod_reconstruct](#), [sod_fPCA](#)

Examples

```
## Not run:
set.seed(2026)
sim <- sim_sod(n = 300, design = "D3", delta = 0.05)
held <- sod_paths(sim$records, method = "held")
mid <- sod_paths(sim$records, method = "midpoint", L = sim$truth$L)
path_eval(held[[1]], c(0.25, 0.5, 0.75))
path_eval(mid[[1]], c(0.25, 0.5, 0.75))

## End(Not run)
```

sod_readings

*Readings of a Send-on-Delta Record***Description**

Returns the reading times and the recorded values of a record: the initial reading at time 0 and one reading per event.

Usage

```
sod_readings(record)
```

Arguments

record An object of class "sod_record".

Value

A data frame with columns time and value, where the values are $v_k = v_0 + \delta(p_1 + \dots + p_k)$.

See Also

[sod_record](#)

Examples

```
## Not run:
set.seed(2026)
sim <- sim_sod(n = 1, design = "D1", delta = 0.05)
sod_readings(sim$records[[1]])

## End(Not run)
```

sod_reconstruct

*Reconstructed Curves Evaluated on a Grid***Description**

Evaluates the held, chord or midpoint reconstructions of a sample of records at given times.

Usage

```
sod_reconstruct(records, t, method = c("held", "chord", "midpoint"), L = NULL)
```

Arguments

records	A list of "sod_record" objects with a common threshold.
t	Evaluation times in $[0, 1]$.
method	Reconstruction: "held", "chord" or "midpoint".
L	Bound on the Lipschitz constant; needed for "midpoint".

Value

A matrix with one row per record and one column per time.

See Also

[sod_paths](#)

Examples

```
## Not run:
## the reconstruction errors of design D4 never exceed delta (held) and
## 2 delta (chord)
set.seed(2026)
grid <- seq(0, 1, length.out = 501)
sim <- sim_sod(n = 100, design = "D4", delta = 0.05, t = grid)
held <- sod_reconstruct(sim$records, grid, method = "held")
chord <- sod_reconstruct(sim$records, grid, method = "chord")
max(abs(held - sim$curves))
max(abs(chord - sim$curves))

## End(Not run)
```

sod_record	<i>Send-on-Delta Records</i>
------------	------------------------------

Description

Creates the send-on-delta record of a curve from its initial value, event times and polarities.

Usage

```
sod_record(initial, times = numeric(0), polarity = numeric(0), delta)
```

```
## S3 method for class sod_record
print(x, ...)
```

Arguments

initial	Initial value $x(0)$.
times	Event times $0 < t_1 < \dots < t_m \leq 1$.
polarity	Polarities, each equal to -1 or 1.
delta	Threshold $\delta > 0$.
x	An object of class "sod_record".
...	Not used.

Details

The record of a continuous curve x is $(x(0), m, t_1, \dots, t_m, p_1, \dots, p_m)$, where t_{k+1} is the first time after t_k at which $|x(t) - x(t_k)| \geq \delta$ and $p_k = (v_k - v_{k-1})/\delta$. The recorded values are $v_k = v_0 + \delta(p_1 + \dots + p_k)$ with $v_0 = x(0)$. Between two events the curve stays in the open band $|x(t) - v_k| < \delta$; this silence is the second source of information in a record. The rule has a hysteresis of width 2δ : a return of the curve to v_k produces no event.

Records are usually produced by [sod_encode](#), [sod_encode_curve](#) or [sim_sod](#).

Value

An object of class "sod_record": a list with components initial, times, polarity and delta.

See Also

[sod_readings](#), [held_path](#), [sod_encode](#)

Examples

```
## Not run:
## a record built by hand
rec <- sod_record(0.2, times = c(0.1, 0.3, 0.7), polarity = c(1, 1, -1), delta = 0.1)
rec
sod_readings(rec)

## the record of Figure 1 of the paper: design D3 with a0 = 0.2, a1 = 0.5
## and a2 = 0.5, recorded with delta = 0.1
rec <- sim_records(cbind(a0 = 0.2, a1 = 0.5, a2 = 0.5), "D3", delta = 0.1)[[1]]
```

```

rec

## End(Not run)

```

sod_width

Integrated Identification Width of a Record

Description

Computes the integral of the distance between the envelopes of a record in closed form.

Usage

```
sod_width(record, L)
```

Arguments

record	An object of class "sod_record", compatible with L.
L	Bound on the Lipschitz constant.

Details

With normalized durations $D_k = L(t_k - t_{k-1})/\delta \geq 1$ between events and $D_* = L(1 - t_m)/\delta$ after the last event,

$$W(o) = \frac{\delta^2}{L} \left(\sum_{k=1}^m \Omega(D_k) + \Omega_*(D_*) \right),$$

where $\Omega(D) = (D-1)(D+3)/4$ for $D \leq 3$ and $2D-3$ otherwise, and $\Omega_*(D) = D^2$ for $D \leq 1$ and $2D-1$ otherwise (Theorem 4.15). The expected width $EW(O)$ is the length of the identified interval of $\int_0^1 \mu(t) dt$. The width depends on where the events fall as well as on how many there are: unused speed budget widens the envelopes more between two events than after the last event.

Value

A number.

See Also

[sod_width_range](#), [sod_envelopes](#)

Examples

```

## Not run:
## mean identification width in design D1 at the three thresholds of the
## paper, with n = 300 and the valid bound L = 3 (experiment E3)
set.seed(2026)
cf <- sim_coefficients(300, "D1")
L <- design_truth("D1")$L
sapply(c(0.10, 0.05, 0.025), function(delta)
  mean(sapply(sim_records(cf, "D1", delta), sod_width, L = L)))

## End(Not run)

```

sod_width_range	<i>Attainable Range of the Integrated Width</i>
-----------------	---

Description

Computes the smallest and the largest integrated widths over all records with a given number of events that are compatible with a Lipschitz bound.

Usage

```
sod_width_range(m, delta, L)
```

Arguments

m	Number of events, at most L/δ .
delta	Threshold $\delta > 0$.
L	Bound on the Lipschitz constant.

Details

Let $\Gamma = L/\delta$ and $x = \Gamma - m$ be the unused speed budget. By Theorem 4.15(iii), $W(o)$ ranges exactly over $[\delta^2\omega_{\min}/L, \delta^2\omega_{\max}/L]$ with $\omega_{\max} = \Omega(x+1)$, and $\omega_{\min}$ equal to x^2 for $x \leq 1/2$, to $(\Gamma+m)^2/(4m+1) - m$ for $1/2 \leq x \leq 2m+1$, and to $2\Gamma - 3m - 1$ beyond. For $0 < x \leq 1/2$ the ratio of the largest to the smallest width is $(x+4)/(4x)$, which is unbounded as L/δ decreases to m .

Value

A named vector with the minimum and the maximum.

See Also

[sod_width](#)

Examples

```
## Not run:
## three events, delta = 0.1 and L = 0.35, as in Figure S1 of the paper
sod_width_range(m = 3, delta = 0.1, L = 0.35) # 0.0071 and 0.0161
## the ratio of the largest to the smallest width as L/delta decreases to 3
x <- c(0.5, 0.25, 0.1, 0.05)
sapply(x, function(s) {
  r <- sod_width_range(m = 3, delta = 0.1, L = 0.1 * (3 + s))
  r[["maximum"]] / r[["minimum"]]
})
(x + 4) / (4 * x)

## End(Not run)
```

Index

*** package**

lebesgueFDA-package, 2

chord_path, 2, 3, 5, 27

design_truth, 2, 4, 11–14

held_path, 2, 3, 5, 7, 27, 30

lebesgueFDA (lebesgueFDA-package), 2

lebesgueFDA-package, 2

midpoint_path, 2, 3, 5, 6, 22, 27

path_eval, 7, 9

path_inner, 8, 9

path_linear, 7, 8, 9, 25

predict.sod_fpca, 10, 24

print.sod_band (sod_band), 14

print.sod_fpca (sod_fpca), 23

print.sod_record (sod_record), 30

sim_coefficients, 4, 11, 12–14

sim_curves, 11, 11, 14

sim_records, 11, 12, 14, 20, 21

sim_sod, 2, 4, 11–13, 13, 30

sod_band, 2, 14, 14, 16, 22, 25, 27

sod_compatible, 16, 22

sod_cov, 17, 24, 26

sod_eigenfunctions, 17, 18, 24

sod_encode, 13, 19, 20, 21, 30

sod_encode_curve, 13, 19, 20, 30

sod_encode_sample, 19, 21

sod_envelopes, 2, 6, 15, 16, 22, 31

sod_fpca, 2, 3, 5, 8, 10, 14, 17, 18, 23, 25–27

sod_linear_bounds, 15, 24

sod_mean, 24, 25

sod_midpoint_risk, 2, 6, 15, 24, 26

sod_paths, 27, 29

sod_readings, 28, 30

sod_reconstruct, 27, 29

sod_record, 2, 19, 28, 30

sod_width, 2, 22, 31, 32

sod_width_range, 31, 32